



AI Business Research & Verification Agent — Complete Build Plan

Chettinad CodeFest 2026 — Grand Finale

Team Lead: Keerthana R

Submission Deadline: 25 June 2026, 10:00 PM

Grand Finale Presentation: 27 June 2026

1. One-Page Summary

What it is: An AI research agent that finds business information from public web sources, but instead of silently picking an answer when sources disagree, it treats every field as a claim that must be supported by evidence before it is trusted.

One-line pitch: Most business research agents collect information. EvidenceIQ treats every piece of information as a claim that must be investigated, challenged, and proven before it is accepted.

Why it stands out: Every team at this finale will scrape Google, Yelp, and directories — that part is table stakes. The differentiator is that EvidenceIQ shows its reasoning: confidence scores that are real probability math (not arbitrary), visible contradiction detection, and an escalation step where the system decides on its own to fetch more evidence before committing to an answer.

Team approach: Solo-led, built with friends helping across parallel modules, over 5 days.

2. Problem Statement (Recap)

Given a category-and-location query (for example, "Cardiologists in Birmingham"), build an agent that discovers every matching business across multiple public sources, collects whatever information is genuinely available online for each one, cross-checks that information across sources rather than trusting any single one, merges duplicate listings of the same business, and produces a structured, ranked report with a transparent summary of how complete and reliable the data is. No invented or estimated data is allowed — only what can actually be found. The system must use free, publicly available sources and open-source tools only, and should be architected to scale from hundreds to tens of thousands of records.

3. Our Approach & Differentiator

The brief itself states the bar: the agent should "behave like a professional researcher rather than a simple web scraper." Every team will satisfy the scraping half of that sentence. EvidenceIQ is built around the second half — the reasoning a real researcher applies before trusting a fact.

Three ideas carry the whole project:

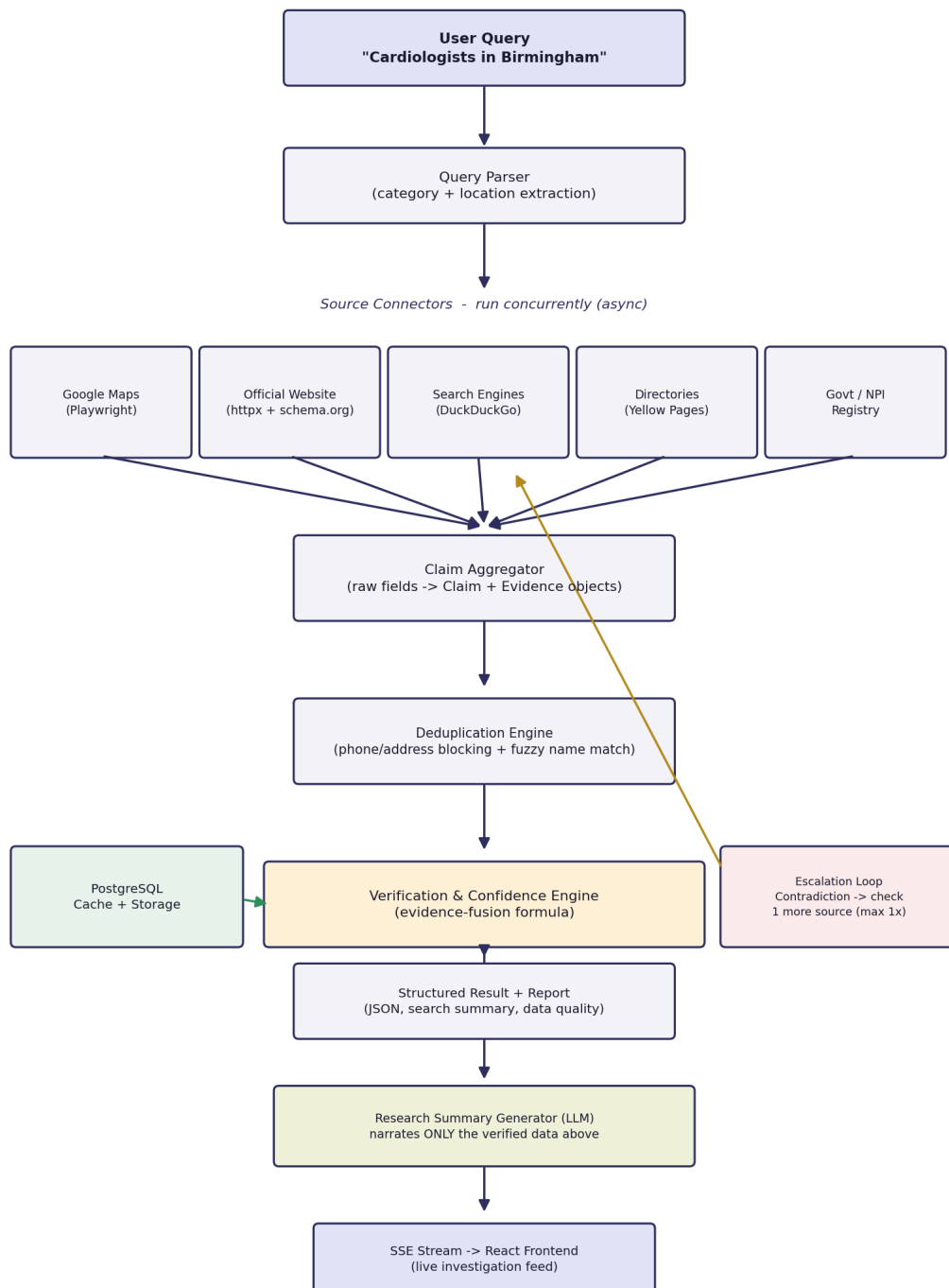
- Every field is a **claim**, not a fact — it only becomes a fact once evidence supports it.
- Confidence scores are **real probability math** from named, explainable factors — never an arbitrary number.
- When evidence conflicts, the system **autonomously fetches one more source** and recalculates, rather than guessing.

One additional claim type beyond simple field-matching is implemented end-to-end as a showcase: cross-checking a business's stated hours (from its official website) against its hours listed on Google Business. This is structured-vs-structured comparison — not open-ended review mining — so it stays reliable for a 5-day build while still demonstrating "claim verification" beyond plain field lookup.

Deliberate scope decision: free-text review mining (e.g., inferring "long wait times" from review sentiment) is named as future work in the pitch, not built live. It would add real NLP complexity without a reliable payoff in the time available.

4. System Architecture

The pipeline is a fixed backbone (every query always passes through every stage) with one adaptive decision point (the Escalation Loop), which is what makes this an agent rather than a script.



Pipeline stages, in order: (1) Query Parser extracts category + location. (2) Source Connectors run concurrently via asyncio. (3) Claim Aggregator converts raw scraped fields into Claim objects with Evidence lists. (4) Deduplication Engine merges records that are the same real-world business. (5) Verification & Confidence Engine computes a confidence score per claim. (6) If a contradiction is found, the Escalation Loop fetches exactly one more source and recomputes (capped at one escalation per claim, to keep runtime and demo behavior predictable). (7) Results are assembled into the structured JSON + summary stats the brief requires. (8) A short LLM call narrates that already-verified JSON into a plain-language research summary — the exact closing step the brief asks for. (9) Everything is cached in PostgreSQL and

streamed live to the frontend over Server-Sent Events (SSE).

Important constraint on stage 8: the LLM is a narrator, not a researcher. It is only allowed to describe numbers, businesses, and verdicts already present in the verified structured output — it never introduces a new claim about a business. This keeps the summary compliant with the brief's "must not invent or estimate information" rule, and is worth stating explicitly in the README so a judge never mistakes it for a second, less-verified data path.

5. Tech Stack

Layer	Technology	Purpose
Backend / API	Python, FastAPI	Async-native; required for concurrent scraping and SSE streaming
Browser automation	Playwright (async)	Scrapes JS-heavy sources: Google Maps results, business listings
HTTP + parsing	httpx, BeautifulSoup	Static page fetches; parses schema.org JSON-LD (free structured data)
Free search	duckduckgo-search (lib)	Discovery + finding official websites/socials, no key needed
Government source	NPI Registry (free public API)	Authoritative healthcare provider data, no key needed
Entity resolution	rapidfuzz, phonenumbers	Fuzzy name matching; phone number normalization (E.164)
Database / cache	PostgreSQL	Structured storage + the caching layer the brief requires
Report narration	One small LLM call (Claude/GPT, or a local model via Ollama)	Narrates the already-verified JSON into a plain-language research summary — grounded, never a new data source
Streaming	Server-Sent Events (SSE)	Pushes each resolved claim to the UI as it is found
Frontend	React	Live investigation feed, evidence checklists, verdict badges
Deployment	Docker Compose	One-command run for judges; hosted link optional per submission rules

No paid APIs anywhere in this stack — every source above is either free/keyless or self-hosted open-source.

6. Data Model: Claim & Evidence

This object is the core of the whole system. Every field of every business is represented this way — the "evidence trail" the demo shows is literally this structure rendered in the UI, not a separate presentation layer.

```
{
  "business_id": "biz_0001",
  "business_name": "ABC Cardiology Center",
  "claims": [
    {
      "field": "phone",
      "value": "205-111-1111",
      "evidence": [
        {"source": "official_website", "supports": true,
         "reliability": 0.85, "freshness": 1.0},
        {"source": "google_business", "supports": true,
         "reliability": 0.75, "freshness": 1.0},
        {"source": "npi_registry", "supports": true,
         "reliability": 0.95, "freshness": 1.0}
      ],
      "confidence": 0.96,
      "refute_score": 0.0,
      "verdict": "Verified"
    },
    {
      "field": "hours_monday",
      "value": "9:00 AM - 5:00 PM",
      "evidence": [
        {"source": "official_website", "supports": true,
         "reliability": 0.85, "freshness": 1.0}
      ],
      "counter_evidence": [
        {"source": "google_business", "value": "Closed after 8 PM",
         "supports": false, "reliability": 0.75}
      ],
      "confidence": 0.62,
      "refute_score": 0.55,
      "verdict": "Contradiction Found",
      "escalation": {
        "triggered": true,
        "checked": "google_business",
        "result": "resolved toward official_website value"
      }
    }
  ]
}
```

7. Confidence Scoring Formula

Avoid arbitrary point systems ("Website = +50, LinkedIn = +25") — a judge will always ask "why those numbers?" and there is no good answer. Instead, treat every corroborating source as independent evidence that reduces uncertainty. This is the same underlying principle as combining independent evidence in Naive Bayes.

For a claim value supported by sources 1..n, each with its own reliability and freshness:

$$\begin{aligned} \text{weight}_i &= \text{reliability}_i \times \text{freshness}_i \\ \text{Confidence} &= 1 - \text{PRODUCT over } i \text{ of } (1 - \text{weight}_i) \end{aligned}$$

Refute_score = same formula, computed over sources that DISAGREE with the value

Worked example — phone number found by three sources, one at a time:

Step	Source added	Reliability	Running confidence
1	Yellow Pages (directory)	0.50	50.0%
2	+ Google Business Profile	0.75	$1 - (0.50 \times 0.25) = 87.5\%$
3	+ Official website	0.85	$1 - (0.125 \times 0.15) = 98.1\%$

Verdict thresholds (applied after confidence and refute_score are both computed):

Condition	Verdict
Confidence ≥ 0.85 and Refute $\leq 0.4 \times \text{Confidence}$	Verified
Refute $\geq 0.4 \times \text{Confidence}$, both ≥ 0.4	Contradiction Found -> triggers escalation
Confidence < 0.5 and Refute < 0.5	Insufficient Evidence
Anything else	Likely (provisional, shown with caveat)

This is the answer ready for "why is your result trustworthy?": every field has a confidence score computed from named, explainable factors, plus a full evidence trail of exactly which sources agreed or disagreed.

8. Source Reliability Weights

Starting weights (0-1 scale), reviewable and adjustable once real data is seen during the build:

Source	Type	Base reliability	Why
Govt registry (e.g. NPI)	Government	0.95	Authoritative, legally maintained record
Official business website	Primary	0.85	Direct from the business, but self-reported
Google Business Profile	Aggregator	0.75	Frequently updated, lightly moderated
schema.org structured data	Primary	0.85	Same trust as the site it's embedded in
LinkedIn (public search results)	Professional network	0.65	Self-reported, updated less often
Yelp	Review platform	0.60	Crowd-checked but can be stale
Yellow Pages	Directory	0.50	Often outdated
Generic/small directory	Directory	0.35	Low verification standards

Freshness decay (applied when serving cached data instead of a fresh scrape):

Cache age	Freshness factor
< 7 days	1.0
7-30 days	0.9
30-90 days	0.75
> 90 days	0.5 (triggers re-scrape)

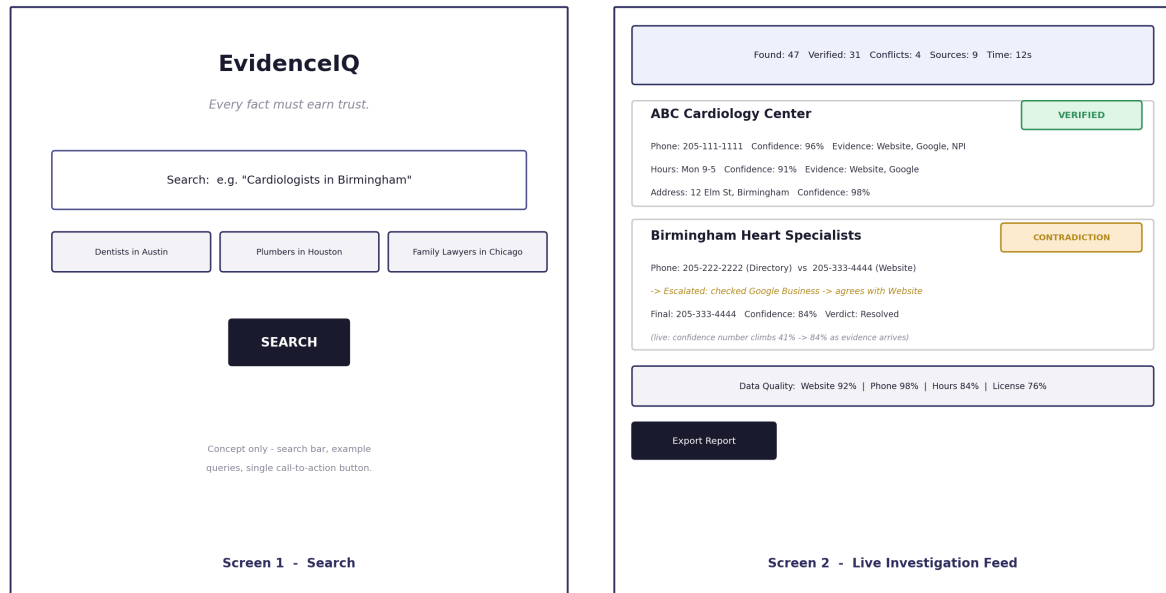
On LinkedIn specifically: only publicly indexed search-result snippets are used for corroboration — the system never logs in or scrapes behind LinkedIn's authentication wall. This keeps the source honest, legal, and easy to defend if a judge asks about it.

9. Deduplication & Entity Resolution

- **Normalize** phone numbers to E.164 format (phonenumbers library) and addresses to a simplified, lowercased, abbreviation-standardized form.
- **Block** candidate records into groups that share a normalized phone OR a normalized address (street + zip) — this is the reliable anchor, since business names for the same entity can vary a lot ("ABC Heart Clinic" vs "ABC Cardiology Center").
- **Confirm** within each block using fuzzy name similarity (rapidfuzz token_sort_ratio) as a secondary signal, not the primary one.
- **Merge** matched records into a single business entity, combining their Claim evidence lists — this merge step is exactly what feeds the confidence formula multiple independent sources for the same field.
- **Edge case:** two distinct businesses sharing one address (e.g., a shared medical building) — if address matches but name similarity is low, keep them separate and flag "co-located, verify manually" rather than wrongly merging them.

10. What The Product Looks Like

Wireframe-level concept (not final styling) of the two screens that matter most for the demo:



Screen 1 — Search: a single search bar with example query chips, one clear call-to-action. No clutter; the product's intelligence shows up after the search, not before it.

Screen 2 — Live Investigation Feed: a running summary bar (found / verified / conflicts / sources / time) updates live. Business cards stream in as they resolve. Each card shows its claims with confidence percentages and a verdict badge (Verified / Contradiction Found / Insufficient Evidence). On a contradiction, the card visibly shows the escalation step happening and the confidence number climbing as more evidence arrives — this live climb is the single most important visual in the whole demo.

Visual theme: a clean "evidence trail" feel (checklists, confidence numbers, color-coded badges) — deliberately not literal courtroom imagery (no gavels, no "prosecution vs defense" framing). The metaphor lives in the language, not the graphics.

11. Day-by-Day Execution Plan (5 Days)

Roles below are generic (You / Friend 1 / Friend 2) — reassign based on who's actually free which day. The one rule that matters: by the end of Day 3, a working end-to-end pipeline must exist on the command line, even if the frontend doesn't exist yet. Everything after that is making it visible and smooth.

Day 1 — Foundations

- **You:** Repo + Docker Compose skeleton (FastAPI + PostgreSQL). Define the Claim/Evidence Pydantic models.
- **Friend 1:** Query Parser — category + location extraction (start with regex/keyword rules; this doesn't need to be fancy).
- **Friend 2:** Source Connector #1 — Google Maps scraper via Playwright (name, address, phone, rating, hours).
- **Goal:** one query runs end-to-end and prints a raw, unverified business list.

Day 2 — More sources + verification core

- **You:** Build the Verification & Confidence Engine (the noisy-OR formula and verdict thresholds from Section 7). This is the most important piece — give it the most careful attention.
- **Friend 1:** Source Connector #2 — official website fetch + schema.org JSON-LD parsing, with plain-text fallback.
- **Friend 2:** Source Connectors #3 and #4 — DuckDuckGo search wrapper + Yellow Pages scraper, plus the NPI Registry connector for the healthcare category.
- **Goal:** four sources feeding evidence into Claim objects for the same business.

Day 3 — Deduplication + escalation + caching + summary

- **You:** Deduplication/entity resolution module (blocking + fuzzy match + merge) from Section 9.
- **Friend 1:** Escalation logic — contradiction detected -> fetch one more source -> recompute, wired into the Verification Engine.
- **Friend 2:** PostgreSQL schema + caching layer (serve cached results instantly, refresh stale entries per the freshness table).
- **Whoever finishes first:** wire the Research Summary Generator — one LLM call that narrates the finished, verified JSON into a short plain-language summary. Keep the prompt strict: it may only describe data already in the JSON, never add a new fact.
- **Goal:** the full backend pipeline works end-to-end on a real test query, producing the structured JSON, the report stats, AND a narrated research summary — matching every item in the brief's required output, not just the data fields.

Day 4 — Streaming, frontend, report

- **You:** FastAPI SSE endpoint streaming claims/businesses to the frontend as they resolve — this is the centerpiece of the demo, worth owning personally.
- **Friend 1:** React frontend — search screen + live streaming results feed + business cards + verdict badges (per Section 10).
- **Friend 2:** Search Summary + Data Quality Summary panels, and the Export Report button (JSON + simple PDF).
- **Goal:** a real working demo on localhost — run a query, watch the investigation unfold live in the browser.

Day 5 — Polish, test, record, rehearse

- **Whole team:** Test three different category/location queries (doctor, lawyer, plumber) to prove the system generalizes; fix what breaks.
- **You:** Find and pre-test ONE query that reliably reproduces a real contradiction — this becomes the primary demo query.
- **Friend 1:** UI polish — styling, badge colors, the confidence-climbing animation.
- **Friend 2:** README, final repo push, one-command Docker Compose run for judges who want to try it themselves.
- **Whole team:** Record the demo video, rehearse the Q&A (Section 13), submit before the deadline.

12. Demo Script

- Open with the one-line pitch (Section 1) before touching the keyboard.
- Run the **pre-tested primary query**. Narrate the confidence number climbing live as sources are checked — this is the moment to slow down, not rush past.
- Point at the contradiction case specifically: "Watch — it didn't just pick one phone number, it noticed the conflict, went and checked another source, and only then committed to an answer."
- Show the Search Summary and Data Quality panel — this directly satisfies the brief's required report format.
- Read the AI-generated research summary out loud — this is the agent closing its own investigation the way a human researcher would write up findings, and it's a natural, memorable way to end the walkthrough.
- Second act: invite the judges to try their own live query. Mention caching here naturally — "even a query we've never seen gets faster next time, which is one of the brief's required features."
- Close by repeating the one-line pitch.

Resilience note: venue wifi during a live, in-person demo is a real risk for live scraping. Keep the pre-tested query cached locally as a guaranteed fallback if live scraping is slow or blocked on the day.

13. Anticipated Q&A

Q: Why should I use this instead of ChatGPT?

A: ChatGPT generates an answer. EvidenceIQ builds an evidence trail and only commits to an answer once it has corroborating proof, with a transparent, reproducible confidence score.

Q: Why is your result trustworthy?

A: Every field has a confidence score computed from named, explainable factors (source reliability, freshness, independent agreement) — never an arbitrary number — plus a full evidence trail of which sources agreed or disagreed.

Q: What actually makes this an "agent" rather than a script?

A: It makes a real decision: when evidence conflicts, it autonomously fetches more evidence and recalculates, instead of running the same fixed sequence regardless of what it finds.

Q: How is the confidence number calculated?

A: 1 minus the product of (1 minus weight) across all agreeing sources — the same principle used to combine independent evidence in Naive Bayes, which treats each source as separate evidence rather than letting one weak source drag down two strong ones.

Q: How does this scale to tens of thousands of records?

A: The architecture is async and source-connectors run concurrently; the caching layer avoids repeat work. Today's demo runs at a representative scale; the design supports adding worker concurrency to scale further.

Q: Why is LinkedIn handled differently from your other sources?

A: We only use publicly indexed search-result snippets for corroboration, never logging in or scraping behind LinkedIn's authentication wall — that keeps the source legitimate and easy to defend.

14. Risk List & Mitigations

Risk	Mitigation
Scrapers get blocked / rate-limited	Caching layer, multiple redundant sources, pre-tested fallback query
Live demo network issues at venue	Local recorded backup + pre-cached primary query
Judge questions the confidence formula	Formula and worked example (Section 7) rehearsed in advance
Scope creep on review-mining claims	Narrowed to one structured claim type (hours cross-check); rest is named as future work
Time overrun across 5 days	Day 3 checkpoint guarantees a working backend even if frontend slips

15. Submission Checklist

- Source code repository link (GitHub)
- Project demonstration recording
- Hosted project link (optional)
- Submit before **25 June 2026, 10:00 PM**
- Prepare to present in person at the Grand Finale on **27 June 2026**